
STYLUS NEXUS: AGENT-DRIVEN ONLINE ARCHIVE BUILDING AND MAINTENANCE

PREPRINT

Xin ZHOU
Ostasiatisches Seminar
Freie Universität Berlin
z.xin@fu-berlin.de

Dingir
Hermes Agent
dingir@agent.qq.com

June 30, 2026

ABSTRACT

Small digital humanities projects face a double burden: source processing, metadata design, public interface work, and long-term maintenance, typically handled by a solo researcher with limited budget and technical support. We introduce Agent Driven Archive, a reusable template that turns source materials into a working online archive through agent collaboration. The template defines a complete pipeline (source intake, OCR, Markdown curation, SQLite export, public presentation) and assigns distinct roles to a building agent, a server maintenance agent, and an in-app semi-agent, while keeping all records source-derived and inspectable. Stylus Nexus, a public archive for Republican-period Daoyuan and spirit-writing materials, is the first instance built from this template. The template-instance design separates reusable workflow from corpus-specific content, enabling solo researchers to build and maintain full-stack digital archives at low cost without starting from scratch.

Keywords digital humanities · online archive · agent workflow · reusable template · source-derived data · sustainability

1 Introduction

Digital humanities archives are often judged by their public interface, but most of their research value is produced before the interface appears. A small archive must collect sources, turn scans into editable text, clean errors, define metadata, generate data exports, and keep a public site running. For a solo researcher, these tasks cross the boundaries of scholarship, software, design, and operations. The real bottleneck is not technical skill alone. It is the absence of a repeatable workflow that connects evidence, interface, and maintenance throughout the project lifecycle.

We built Agent Driven Archive to fill this gap. It is a reusable workflow repository: a workspace skeleton, Markdown templates, validation and export scripts, search index generation, OCR helpers, skill examples, and a demo renderer, organized around a layered architecture for agent collaboration (Zhou, 2026). Stylus Nexus is the first instance built from this template, a public archive of Republican-period Daoyuan, Fuji practice, and spirit-writing publications. It contains scanned PDFs, OCR Markdown, cleaned and translated records, research essays, timeline and map data, SQLite outputs, and a Next.js public application, all produced by following the template’s pipeline. The relationship is explicit: Agent Driven Archive defines the reusable workflow; Stylus Nexus instantiates it with a specific corpus and demonstrates that the workflow reaches a working public interface.

The design question driving this work is: how can a small digital humanities project deeply integrate AI agents into its workflow under limited budgets and technical skill, while maintaining long-term usability? Our answer is a source-derived agent workflow. Starting from raw materials, the researcher and agents collaborate to accelerate project building and maintenance. Raw sources enter the system first; Markdown and YAML hold editable intermediate records; SQLite and search indexes are generated outputs; the public interface reads from those outputs. AI assistance is constrained by selection, route, retrieval, and citation context.

We make two contributions. First, a “2.5 Agents” architecture that assigns building, server, and semi-agent roles within a source-derived pipeline. Second, a template-instance design pattern for digital humanities: a transferable workflow repository separated from corpus-specific data, with reusability demonstrated by a working instance.

2 Related Work and Design Position

Our design follows a view of digital humanities in which tools and interfaces are not neutral containers. Galey and Ruecker argue that a prototype can make an argument through its structure and affordances (Galey and Ruecker, 2010). We apply this principle at the workflow level: the more transferable object is not the public site but the chain that links sources, editorial actions, machine-readable data, and user interaction.

We also follow a humanistic view of data. Drucker warns that humanities materials contain ambiguity, uncertainty, and interpretive pressure that simple information design flattens (Drucker, 2011). We address this by making Markdown the editable middle layer. OCR output is not published as final truth; through manual researcher intervention, cleaned records carry tags, notes, page anchors, and review status. The public interface stays close to sources while preserving editorial judgment, greatly improving credibility and usability.

Digital preservation offers a second frame. Owens describes preservation as an ongoing craft joining technical care with organizational practice (Owens, 2018). We do not claim institutional-scale preservation. Our practical claim is narrower: small projects need a way to separate raw materials, editable records, generated outputs, and runtime services. That separation makes migration and review easier. Recent FAIR corpus management guidelines similarly identify open formats, standardized metadata, version-controlled documentation, and archival licensing as minimal infrastructure for sustainable digital corpora (Alieva et al., 2024).

The AI component sits within this preservation and provenance frame. Retrieval-augmented generation joins a generator with retrieved evidence (Lewis et al., 2020), and graph-aware extensions such as LightRAG offer further capabilities for multi-hop discovery (Guo et al., 2024). In our architecture, however, these are optional modules, not core dependencies. A project launches with direct user-attached selections and local full-text search; it adds stronger retrieval only when corpus scale and hardware justify the cost, enhancing the built-in semi-agent’s capabilities on demand.

Our work is also close to minimal computing, which values small, maintainable, and locally appropriate systems over maximal infrastructure (Digital Humanities Climate Coalition, 2025). We adopt this value without rejecting modern frameworks or AI tools. We simply ask each tool to justify its maintenance cost. The result is a layered system where costly parts are optional, replaceable, or deferred.

3 Project Overview

Agent Driven Archive is a workflow and starter repository. As the developer of both repositories, we derived an extraction map by identifying which components of the Stylus Nexus build were reusable across topics and which were tied to the Daoyuan corpus. This map sorts source-project parts into three categories: reusable now, reusable with refactor, and project-specific. It records what can travel from one archive to another and is a key output of this project.

The template defines a canonical pipeline: external sources → OCR Markdown → cleaned, tagged, or translated Markdown → domain SQLite → public UI and API. Data structures separate Archive, Blog, Timeline Map, and Agent Workspace materials. Stylus Nexus follows this pipeline to produce a public archive for Republican-period Daoyuan materials. Its local archive database currently contains four viewer sources, 166 text chunks, and 573 viewer pages; the timeline database holds 163 reviewed places and 163 timeline events. These counts are modest, but they are sufficient to exercise the pipeline across content types and confirm that the output is a working public application with catalogue, viewer, blog, timeline map, and AI-assisted reading.

4 Architecture

The core architecture is a five-layer source-derived pipeline. Raw materials enter at the Source layer and remain separate from the public site. At the Editorial layer, OCR and cleaned or translated Markdown stay human-readable and agent-editable. The Export layer generates manifests, SQLite databases, search indexes, and viewer assets. The Application layer renders catalogue, viewer, blog, timeline map, and search modules. The Agent layer runs across all stages: a building agent handles code and data workflow tasks, a server agent manages maintenance operations, and a semi-agent embedded in the public interface assists readers with bounded context.

Layer	Role
Source	Raw scans, PDFs, images, and source inventory kept separate from the public site.
Editorial	OCR and cleaned or translated Markdown remain human-readable and agent-editable.
Export	Scripts generate manifests, SQLite databases, search indexes, and viewer assets.
Application	Web app renders catalogue, viewer, blog, timeline map, and search modules.
Agent	Building, server, and semi-agent roles support construction, maintenance, and reader assistance.

The division of agent labor is central to the design. The building agent sets up repositories, writes scripts, repairs data exports, and implements public routes. The server agent reviews contribution queues, sends mail digests, runs scheduled exports, and reports maintenance status. The semi-agent supports reading, translation, citation, and contextual explanation inside the public interface, but its context is bounded by the active page, selected text, and local retrieval. This division avoids two risks we observed in early prototypes: over-agentification (every task becoming its own autonomous role) and the generic chatbot problem (an archive with an unmoored AI widget). The semi-agent is document-first: selected text is normalized and attached to requests, local full-text search notes are added only when selection context exists, and the prompt instructs the assistant to treat attached selections as primary evidence and avoid invented citations.

Advanced retrieval is an optional extension, not a core requirement. The dependency matrix separates required core components (workspace validation, manifest export, search index generation) from deferred modules such as local embeddings or LightRAG. A project can launch and serve users with direct selection context and local full-text search alone, and adopt stronger retrieval methods later without restructuring the pipeline.

5 Transferable Workflow

A finished archive site alone is hard to reuse because it mixes corpus knowledge, design choices, scripts, routes, and deployment assumptions. We solve this by extracting the methodological skeleton from the first instance, distinguishing general workflow from technical scaffolding from corpus-specific content. This is a stronger form of reuse than sharing a template page: it shares a way of making archives.

Category	Transfer decision
Reusable now	Domain split, workspace skeleton, Markdown templates, source anchors, source inventory, contribution queue pattern.
Reusable with refactor	Export scripts, archive viewer adapters, citation utilities, AI sidebar injection flow, timeline normalization.
Project-specific	Daoyuan prose, branded visuals, fixed corpus assumptions, acknowledgments, course-specific framing.
Deferred extension	Generic domain export package, stable citation helper, server agent hosting, deployment kit.

The template does not simply use AI to generate pages. It defines a chain of archival tasks with explicit review points at each stage. The same workflow can accept another corpus because the strongest assumptions are about work order and data boundaries, not about any particular topic. A project about book catalogues, manuscript collections, or local archive materials can keep the same source inventory structure, Markdown templates, export stages, and public modules while replacing the specific content.

The workflow supports gradual adoption. A researcher can start with the workflow-only profile (Git, Node.js, Python, Markdown, YAML, JSON, and basic OCR helpers), which already covers intake, cleaning, and export. For researchers who prefer low-code content management, the editorial layer integrates naturally with Obsidian: the workspace is a plain Markdown folder structure that Obsidian opens directly, and its Dataview and Templater plugins provide visual querying and templating for source records, essays, events, and relationship data. Agents read and write the same Markdown files, so the human workspace and the agent workspace are the same workspace, forming a real-time closed loop without custom UI. The archive site profile adds a public app, PDF rendering, search, and map modules. The retrieval-enhanced profile optionally adds embeddings or LightRAG. This progression matters because digital humanities projects often fail when forced to adopt a full infrastructure stack before the corpus is ready, and grant-funded projects frequently struggle to survive beyond their initial support window (Casey, 2023). By templating most technical dependencies and enabling researcher-agent collaboration, we lower both the initial building threshold and the long-term maintenance burden. A solo researcher working with agents can complete a full-stack project that far exceeds static-site-generator results, at a cost structure where only server hosting and AI token consumption remain as recurring expenses.

6 Implementation: The Stylus Nexus Instance

Stylus Nexus implements the application layer as a Next.js and React application with TypeScript, Tailwind CSS, Leaflet, Markdown rendering, and SQLite via Node. The route structure covers home page, archive catalogue, archive viewer, blog, timeline map, contribution form, search overlay, and AI sidebar. The archive viewer links PDF page images with OCR text; the timeline map derives structured event and place data from source tables. These modules confirm that the workflow reaches a working public research interface, not merely a data preparation pipeline.

The data layer follows the source-derived boundary. All public records trace back to Markdown files or structured event records, never to unverified generated text. The archive database stores source records, text chunks, viewer sources, and viewer pages; the timeline database stores date conversions, event and place candidates, reviewed places, feature links, and timeline events.

The public AI route uses the OpenRouter platform’s Free AI API, with per-IP daily limits and concurrency caps to contain token consumption; the server agent monitors usage in the background to prevent abuse. Translation mode is allowed only when a selection is attached. Local retrieval is scoped by route, with archive and blog types treated separately. These measures lower hallucination risk by keeping context visible in the prompt and citations list, without requiring embeddings infrastructure at launch.

Portability is enforced by scripts and the dependency matrix. Scripts validate workspace structure, export source manifests, build search indexes, render a static demo site, initialize contribution storage, and review contribution queues. The dependency matrix distinguishes required tools from optional modules (Leaflet, OCR rendering, local PDF translation, embeddings, LightRAG), so that a new project knows exactly which components are essential and which are enhancements. This clean separation is what makes low-cost migration from one corpus to another feasible.

7 Contributions and Limitations

Our first contribution is a practical model of agent-driven archival work. AI agents are not authors of evidence but collaborators in a controlled workflow. The building agent accelerates system construction, the server agent supports maintenance, and the semi-agent helps public users read and reuse the archive. Making these roles legible is a prerequisite for scholarly trust.

Our second contribution is the source-derived data boundary. Every public record must trace back to a source file, a cleaned Markdown document, a translation, or a structured event record. Generated SQLite files are outputs, not editorial ground truth. This rule lets a researcher use agents for speed while preserving the ability to inspect and repair the chain of evidence at any point.

Our third contribution is the template-instance design pattern. Many digital humanities projects publish final sites; far fewer publish the working path by which another corpus can reproduce the result. Agent Driven Archive does this by separating general workflow from topic-specific data, branded design, and corpus assumptions. The extraction map is not merely documentation: it is a method for deciding what is reusable and what must stay local. Stylus Nexus demonstrates that the template produces a working public archive with real data while remaining visibly decoupled from the template repository.

There are limitations. The project has not been evaluated through a formal user study. Some export scripts remain corpus-specific and need refactoring. The live instance reflects hardware and hosting constraints. A second instance built by an independent researcher would provide stronger evidence of reusability. These limits do not weaken our main claim but define its scope: we describe a tested, documented, and single-instance-validated architecture, not a finished platform standard.

8 Conclusion

Agent Driven Archive demonstrates that a small digital humanities archive can be built as both a reusable template and a public site. Its core value is the connection between source-derived data, agent collaboration, and constrained AI assistance, packaged in a repository that separates workflow from content. Stylus Nexus proves the template works in practice: it produces a live archive with real materials, while the template itself stays clean enough for another project to adopt. The model is straightforward: keep sources and editable records separate from generated data, assign agents to real work roles, expose a public interface, add retrieval capabilities in stages, and publish the workflow as a transferable artefact. This design gives small teams a practical path toward low-cost, sustainable online archives, without hiding the labor of curation behind a generic AI layer.

Data Availability

The reusable workflow repository is available at <https://github.com/KimFischer99/Agent-Driven-Archive>. The Stylus Nexus project materials used in this work were inspected in the local project folder. Public project access is listed at <https://stylusnexus.localhost.cc>.

AI Disclosure

This work was prepared with assistance from AI-powered academic writing and formatting tools. The author directed the argument, selected the project evidence, reviewed the claims, and takes full responsibility for the accuracy and integrity of the work.

References

- Alieva, A., Bozan, J. S., Müller-Laackman, J., and Kudela, M. X. (2024). HOW TO BE FAIR? – managing and archiving digital corpora – theoretical workflows and practical examples. Blog post, *Closing the Gap in Non-Latin-Script Data*. November 14, 2024. Accessed June 30, 2026.
- Casey, C. (2023). Building digital projects to outlive their funding. *AVAR: An Interdisciplinary Journal of Life and Society in the Ancient Near East*, 2(2):355–378.
- Digital Humanities Climate Coalition (2025). Minimal computing. DHCC Toolkit. Last modified April 30, 2025. Accessed June 30, 2026.
- Drucker, J. (2011). Humanities approaches to graphical display. *Digital Humanities Quarterly*, 5(1):000091.
- Galey, A. and Ruecker, S. (2010). How a prototype argues. *Literary and Linguistic Computing*, 25(4):405–424.
- Guo, Z., Xia, L., Yu, Y., Ao, T., and Huang, C. (2024). Lightrag: Simple and fast retrieval-augmented generation. arXiv preprint, arXiv:2410.05779.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W.-t., Rocktaschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. arXiv preprint, arXiv:2005.11401.
- Owens, T. (2018). *The Theory and Craft of Digital Preservation*. Johns Hopkins University Press, Baltimore, MD.
- Zhou, X. (2026). Agent driven archive. GitHub repository. Accessed June 30, 2026.